

SQL Interview Preparation

Basic to Advanced — Full Syntax, Joins, Subqueries & Most Asked Questions

A complete reference with runnable code snippets, sample-data dry runs, and the most frequently asked SQL interview questions with solutions.

Table of Contents

- 1. Introduction & Sample Database Used Throughout
- 2. DDL — CREATE, ALTER, DROP, TRUNCATE, Data Types
- 3. Constraints (PK, FK, UNIQUE, CHECK, DEFAULT, NOT NULL)
- 4. DML — INSERT, UPDATE, DELETE, MERGE
- 5. DQL Basics — SELECT, WHERE, DISTINCT, ORDER BY, LIMIT
- 6. Operators & Filtering — AND/OR, BETWEEN, IN, LIKE, IS NULL
- 7. Aggregate Functions, GROUP BY & HAVING
- 8. Joins — Inner, Left, Right, Full, Cross, Self (all with dry runs)
- 9. Subqueries — Scalar, Multi-row, Correlated, Nested
- 10. Set Operations — UNION, UNION ALL, INTERSECT, EXCEPT
- 11. Window Functions — ROW_NUMBER, RANK, LEAD/LAG, SUM OVER
- 12. Common Table Expressions (CTE) & Recursive CTE
- 13. Views
- 14. Indexes & Query Performance
- 15. Stored Procedures & User Defined Functions
- 16. Triggers
- 17. Transactions & ACID Properties
- 18. Normalization (1NF – BCNF)
- 19. String, Date & Numeric Functions
- 20. Most Repeated SQL Interview Questions (25 Q&A)
- 21. Clause Execution Order & Final Cheat Sheet

1. Introduction & Sample Database

SQL (Structured Query Language) is used to create, read, update and delete data in a relational database. This guide uses two small, consistent sample tables throughout so every join, subquery, and window function example can be dry-run by hand.

Employees Table

emp_id	emp_name	salary	dept_id	manager_id	hire_date
1	Alice	90000	10	NULL	2020-01-15
2	Bob	75000	10	1	2021-03-10
3	Carol	85000	20	1	2019-07-01
4	David	60000	20	3	2022-05-20
5	Eve	75000	30	3	2021-11-30
6	Frank	55000	NULL	1	2023-02-14

Departments Table

dept_id	dept_name	location
10	Engineering	Bangalore
20	Sales	Mumbai
30	Marketing	Delhi
40	HR	Pune

Notice: Frank has `dept_id = NULL` (no department assigned yet), and department 40 (HR) has no employees. These gaps are intentional — they are exactly what makes JOIN and subquery examples meaningful.

Setup Script (used to create these tables)

```
CREATE TABLE Departments (  
  dept_id    INT PRIMARY KEY,  
  dept_name  VARCHAR(50),  
  location   VARCHAR(50)  
);  
  
CREATE TABLE Employees (  
  emp_id     INT PRIMARY KEY,  
  emp_name   VARCHAR(50),  
  salary     DECIMAL(10,2),  
  dept_id    INT,  
  manager_id INT,  
  hire_date  DATE,  
  FOREIGN KEY (dept_id) REFERENCES Departments(dept_id),  
  FOREIGN KEY (manager_id) REFERENCES Employees(emp_id)  
);  
  
INSERT INTO Departments VALUES  
  (10,'Engineering','Bangalore'), (20,'Sales','Mumbai'),  
  (30,'Marketing','Delhi'), (40,'HR','Pune');  
  
INSERT INTO Employees VALUES  
  (1,'Alice',90000,10,NULL,'2020-01-15'),  
  (2,'Bob',75000,10,1,'2021-03-10'),  
  (3,'Carol',85000,20,1,'2019-07-01'),  
  (4,'David',60000,20,3,'2022-05-20'),  
  (5,'Eve',75000,30,3,'2021-11-30'),  
  (6,'Frank',55000,NULL,1,'2023-02-14');
```

2. DDL — Data Definition Language

DDL statements define and modify the structure of database objects. They are auto-committed in most databases.

2.1 CREATE

```
CREATE DATABASE company;

CREATE TABLE Employees (
  emp_id      INT PRIMARY KEY,
  emp_name    VARCHAR(50) NOT NULL,
  salary      DECIMAL(10,2) DEFAULT 0,
  dept_id     INT
);

-- Create table from another table's result (structure + data)
CREATE TABLE HighEarnings AS
SELECT * FROM Employees WHERE salary > 80000;
```

2.2 ALTER

```
ALTER TABLE Employees ADD COLUMN email VARCHAR(100);
ALTER TABLE Employees DROP COLUMN email;
ALTER TABLE Employees MODIFY COLUMN salary DECIMAL(12,2); -- MySQL
ALTER TABLE Employees ALTER COLUMN salary TYPE NUMERIC(12,2); -- PostgreSQL
ALTER TABLE Employees RENAME COLUMN emp_name TO full_name;
ALTER TABLE Employees ADD CONSTRAINT fk_dept
  FOREIGN KEY (dept_id) REFERENCES Departments(dept_id);
```

2.3 DROP vs TRUNCATE vs DELETE

Command	Removes	Rollback?	Resets identity	Fires triggers	Type
DROP TABLE	Table structure + data	No (in most DBs)	N/A	No	DDL
TRUNCATE TABLE	All rows only, keeps structure	No (usually)	Yes	No	DDL
DELETE FROM table	Rows (optionally filtered by WHERE)	Yes	No	Yes	DML

Interview favorite: "Difference between DELETE, TRUNCATE and DROP" — remember: DELETE is DML (row-by-row, logged, rollback-able, supports WHERE); TRUNCATE is DDL (deallocates data pages, cannot use WHERE, faster, resets auto-increment); DROP removes the entire object from the schema.

2.4 Common Data Types

Category	Types
Numeric	INT, BIGINT, SMALLINT, DECIMAL(p,s), NUMERIC, FLOAT, DOUBLE
String	CHAR(n), VARCHAR(n), TEXT
Date/Time	DATE, TIME, DATETIME, TIMESTAMP
Boolean	BOOLEAN / BIT / TINYINT(1)
Other	JSON, BLOB, ENUM, UUID

3. Constraints

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL + UNIQUE	<code>emp_id INT PRIMARY KEY</code>
FOREIGN KEY	Links to a PK in another table; enforces referential integrity	<code>FOREIGN KEY(dept_id) REFERENCES Departments(dept_id)</code>
UNIQUE	All values in column must be distinct (NULLs allowed)	<code>email VARCHAR(100) UNIQUE</code>
NOT NULL	Column cannot store NULL	<code>emp_name VARCHAR(50) NOT NULL</code>
CHECK	Value must satisfy a boolean condition	<code>CHECK (salary > 0)</code>
DEFAULT	Value used when none is supplied	<code>status VARCHAR(10) DEFAULT 'ACTIVE'</code>

```
CREATE TABLE Employees (  
  emp_id    INT PRIMARY KEY,  
  emp_name  VARCHAR(50) NOT NULL,  
  salary    DECIMAL(10,2) CHECK (salary > 0),  
  dept_id   INT,  
  status    VARCHAR(10) DEFAULT 'ACTIVE',  
  CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)  
);
```

Interview favorite: "Can a table have more than one PRIMARY KEY?" No — only one PRIMARY KEY per table, but it can be a **composite key** spanning multiple columns: `PRIMARY KEY (order_id, product_id)`. A table CAN have multiple UNIQUE constraints, unlike PRIMARY KEY.

4. DML — Data Manipulation Language

4.1 INSERT

```
INSERT INTO Employees (emp_id, emp_name, salary, dept_id, manager_id, hire_date)
VALUES (7, 'Grace', 68000, 10, 1, '2023-06-01');

-- Multi-row insert
INSERT INTO Employees (emp_id, emp_name, salary, dept_id)
VALUES (8, 'Heidi', 71000, 20), (9, 'Ivan', 66000, 30);

-- Insert from another query
INSERT INTO HighEarnings SELECT * FROM Employees WHERE salary > 80000;
```

4.2 UPDATE

```
UPDATE Employees
SET salary = salary * 1.10
WHERE dept_id = 10;

-- Update using a join (MySQL / SQL Server style)
UPDATE Employees e
JOIN Departments d ON e.dept_id = d.dept_id
SET e.salary = e.salary + 5000
WHERE d.location = 'Bangalore';
```

Always test UPDATE/DELETE with a SELECT using the same WHERE clause first — omitting WHERE updates or deletes every row in the table.

4.3 DELETE

```
DELETE FROM Employees WHERE emp_id = 9;

-- Delete rows matching a subquery
DELETE FROM Employees
WHERE dept_id IN (SELECT dept_id FROM Departments WHERE location = 'Pune');
```

4.4 MERGE (UPSERT)

Combines INSERT + UPDATE in one statement — insert if not found, update if it exists.

```
MERGE INTO Employees AS target
USING Staging AS source
ON target.emp_id = source.emp_id
WHEN MATCHED THEN
    UPDATE SET target.salary = source.salary
WHEN NOT MATCHED THEN
    INSERT (emp_id, emp_name, salary, dept_id)
    VALUES (source.emp_id, source.emp_name, source.salary, source.dept_id);

-- MySQL equivalent
INSERT INTO Employees (emp_id, emp_name, salary)
VALUES (2, 'Bob', 80000)
ON DUPLICATE KEY UPDATE salary = VALUES(salary);

-- PostgreSQL equivalent
INSERT INTO Employees (emp_id, emp_name, salary)
VALUES (2, 'Bob', 80000)
ON CONFLICT (emp_id) DO UPDATE SET salary = EXCLUDED.salary;
```

5. DQL Basics — SELECT

5.1 Basic SELECT, DISTINCT, ORDER BY, LIMIT

```
SELECT emp_name, salary FROM Employees;

SELECT DISTINCT dept_id FROM Employees;      -- unique dept_ids only

SELECT * FROM Employees ORDER BY salary DESC;  -- highest first
SELECT * FROM Employees ORDER BY dept_id ASC, salary DESC;

-- Row limiting (syntax differs by database)
SELECT * FROM Employees ORDER BY salary DESC LIMIT 3;      -- MySQL / PostgreSQL
SELECT TOP 3 * FROM Employees ORDER BY salary DESC;        -- SQL Server
SELECT * FROM Employees ORDER BY salary DESC FETCH FIRST 3 ROWS ONLY; -- Oracle/ANSI

-- Pagination
SELECT * FROM Employees ORDER BY emp_id LIMIT 5 OFFSET 10;  -- rows 11-15
```

Dry Run: `SELECT DISTINCT dept_id FROM Employees;`

Raw dept_id column: 10, 10, 20, 20, 30, NULL → distinct result set (order not guaranteed):

dept_id
10
20
30
NULL

5.2 Column Aliases & Expressions

```
SELECT emp_name AS name,
       salary,
       salary * 12 AS annual_salary,
       salary * 0.10 AS bonus
FROM Employees;
```

6. Operators & Filtering

6.1 Comparison & Logical Operators

```
SELECT * FROM Employees WHERE salary > 70000;
SELECT * FROM Employees WHERE salary >= 60000 AND dept_id = 20;
SELECT * FROM Employees WHERE dept_id = 10 OR dept_id = 30;
SELECT * FROM Employees WHERE NOT dept_id = 10;
```

6.2 BETWEEN, IN, LIKE, IS NULL

```
-- BETWEEN is inclusive on both ends
SELECT * FROM Employees WHERE salary BETWEEN 60000 AND 85000;

-- IN checks membership in a list (or subquery)
SELECT * FROM Employees WHERE dept_id IN (10, 30);

-- LIKE: % = any sequence of chars, _ = exactly one char
SELECT * FROM Employees WHERE emp_name LIKE 'A%';    -- starts with A
SELECT * FROM Employees WHERE emp_name LIKE '%e';    -- ends with e
SELECT * FROM Employees WHERE emp_name LIKE '_a%';    -- 2nd letter is a

-- NULL handling — always use IS NULL / IS NOT NULL, never = NULL
SELECT * FROM Employees WHERE manager_id IS NULL;
SELECT * FROM Employees WHERE dept_id IS NOT NULL;
```

Dry Run: WHERE salary BETWEEN 60000 AND 85000

emp_name	salary	Included?
Alice	90000	No (above 85000)
Bob	75000	Yes
Carol	85000	Yes (inclusive)
David	60000	Yes (inclusive)
Eve	75000	Yes
Frank	55000	No (below 60000)

Interview favorite: "Why does WHERE salary = NULL return nothing?" Because NULL means "unknown" — any comparison with NULL (=, <>, >, <) evaluates to UNKNOWN, not TRUE, so no rows match. Only IS NULL / IS NOT NULL correctly test for NULL.

7. Aggregate Functions, GROUP BY & HAVING

7.1 Aggregate Functions

Function	Purpose
COUNT(*)	Number of rows (including NULLs)
COUNT(col)	Number of non-NULL values in col
SUM(col)	Total of numeric column
AVG(col)	Average of numeric column
MIN(col) / MAX(col)	Smallest / largest value

```
SELECT COUNT(*) AS total_employees FROM Employees;
SELECT COUNT(dept_id) AS assigned_employees FROM Employees;  -- excludes Frank (NULL)
SELECT AVG(salary) AS avg_salary FROM Employees;
SELECT MAX(salary) - MIN(salary) AS salary_range FROM Employees;
```

7.2 GROUP BY

```
SELECT dept_id, COUNT(*) AS emp_count, AVG(salary) AS avg_salary
FROM Employees
GROUP BY dept_id;
```

Dry Run

dept_id	emp_count	avg_salary
10	2	82500 (Alice 90000, Bob 75000)
20	2	72500 (Carol 85000, David 60000)
30	1	75000 (Eve)
NULL	1	55000 (Frank)

7.3 HAVING vs WHERE

WHERE filters rows *before* grouping; **HAVING** filters groups *after* aggregation. You cannot use an aggregate function inside WHERE.

```
-- Departments with more than 1 employee
SELECT dept_id, COUNT(*) AS emp_count
FROM Employees
GROUP BY dept_id
HAVING COUNT(*) > 1;

-- Combine both: filter rows first, then filter groups
SELECT dept_id, AVG(salary) AS avg_salary
FROM Employees
WHERE hire_date >= '2020-01-01'
GROUP BY dept_id
HAVING AVG(salary) > 65000;
```

Interview favorite: "WHERE vs HAVING" — WHERE works on individual rows before GROUP BY; HAVING works on aggregated groups after GROUP BY. Using an aggregate in WHERE (e.g. WHERE COUNT(*) > 1) throws an error.

8. Joins — Inner, Left, Right, Full, Cross, Self

All join examples use the Employees / Departments tables from Section 1. Recall: Frank has `dept_id = NULL`, and department 40 (HR) has zero employees.

8.1 INNER JOIN

Returns only rows where the join condition matches in **both** tables.

```
SELECT e.emp_name, e.salary, d.dept_name
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.dept_id;
```

Dry Run Result

emp_name	salary	dept_name
Alice	90000	Engineering
Bob	75000	Engineering
Carol	85000	Sales
David	60000	Sales
Eve	75000	Marketing

Frank (dept_id NULL) and HR (dept_id 40, no employees) are both dropped — 5 rows only.

8.2 LEFT JOIN (LEFT OUTER JOIN)

Returns **all** rows from the left table, plus matches from the right; unmatched right columns become NULL.

```
SELECT e.emp_name, e.dept_id, d.dept_name
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Dry Run Result

emp_name	dept_id	dept_name
Alice	10	Engineering
Bob	10	Engineering
Carol	20	Sales
David	20	Sales
Eve	30	Marketing
Frank	NULL	NULL

All 6 employees appear; Frank's dept_name is NULL since he has no matching department.

8.3 RIGHT JOIN (RIGHT OUTER JOIN)

Mirror of LEFT JOIN — returns **all** rows from the right table, plus matches from the left.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d ON e.dept_id = d.dept_id;
```

Dry Run Result

emp_name	dept_name
Alice	Engineering
Bob	Engineering
Carol	Sales
David	Sales
Eve	Marketing
NULL	HR

HR appears with NULL emp_name because no employee belongs to it. (RIGHT JOIN is rarely used in practice — the same result can be written as a LEFT JOIN by swapping table order.)

8.4 FULL OUTER JOIN

Returns all rows from both tables; unmatched columns from either side are NULL. Equivalent to LEFT JOIN UNION RIGHT JOIN.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
FULL OUTER JOIN Departments d ON e.dept_id = d.dept_id;

-- MySQL has no FULL OUTER JOIN — emulate with UNION
SELECT e.emp_name, d.dept_name FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name FROM Employees e RIGHT JOIN Departments d ON e.dept_id = d.dept_id;
```

Dry Run Result

emp_name	dept_name
Alice	Engineering
Bob	Engineering
Carol	Sales
David	Sales
Eve	Marketing
Frank	NULL
NULL	HR

7 rows total — both the unmatched employee (Frank) and the unmatched department (HR) are kept.

8.5 CROSS JOIN

Cartesian product — every row of table A paired with every row of table B. No ON condition. Result size = rows(A) × rows(B).

```
SELECT e.emp_name, d.dept_name
FROM Employees e
CROSS JOIN Departments d;
-- 6 employees x 4 departments = 24 rows
```

8.6 SELF JOIN

A table joined with itself — used for hierarchical data like employee-manager relationships.

```
SELECT emp.emp_name AS employee, mgr.emp_name AS manager
FROM Employees emp
LEFT JOIN Employees mgr ON emp.manager_id = mgr.emp_id;
```

Dry Run Result

employee	manager
Alice	NULL
Bob	Alice
Carol	Alice
David	Carol
Eve	Carol
Frank	Alice

8.7 Multiple Joins (3+ tables)

Chain joins to pull data from more than two tables. Suppose a Projects table exists: Projects(project_id, project_name, dept_id).

```
SELECT e.emp_name, d.dept_name, p.project_name
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
JOIN Projects p      ON d.dept_id = p.dept_id
WHERE p.project_name IS NOT NULL
ORDER BY d.dept_name, e.emp_name;
```

Tip: When joining 3+ tables, join the tables that share a direct foreign key relationship first, then extend outward. Always alias tables (e, d, p) to keep column references short and unambiguous.

8.8 Joins with Aggregation

```
-- Department-wise employee count and total salary (departments with 0 employees still shown)
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count, COALESCE(SUM(e.salary), 0) AS total_salary
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Dry Run Result

dept_name	emp_count	total_salary
Engineering	2	165000
Sales	2	145000
Marketing	1	75000
HR	0	0

Interview favorite: "Find departments with no employees" — this requires a LEFT JOIN (Departments as the left/driving table) followed by WHERE e.emp_id IS NULL, not an INNER JOIN.

```
SELECT d.dept_name
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
-- Result: HR
```

8.9 Join Types Summary

Join	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All left rows + matched right rows (NULL if no match)
RIGHT JOIN	All right rows + matched left rows (NULL if no match)
FULL OUTER JOIN	All rows from both, matched where possible
CROSS JOIN	Cartesian product — every combination
SELF JOIN	Table joined to itself (via alias)

9. Subqueries

A subquery is a query nested inside another query. It can appear in `SELECT`, `FROM`, or `WHERE` clauses.

9.1 Scalar Subquery (returns a single value)

```
-- Employees earning above the company average
SELECT emp_name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Dry Run

Inner query: `AVG(salary)` = (90000+75000+85000+60000+75000+55000)/6 = 73333.33

emp_name	salary
Alice	90000
Bob	75000
Carol	85000
Eve	75000

9.2 Subquery in SELECT clause

```
SELECT emp_name, salary,
       (SELECT AVG(salary) FROM Employees) AS company_avg,
       salary - (SELECT AVG(salary) FROM Employees) AS diff_from_avg
FROM Employees;
```

9.3 Multi-row Subquery — IN / ANY / ALL

```
-- IN: match any value in the subquery's result set
SELECT emp_name FROM Employees
WHERE dept_id IN (SELECT dept_id FROM Departments WHERE location IN ('Bangalore','Mumbai'));

-- ANY: true if condition holds for at least one returned value
SELECT emp_name, salary FROM Employees
WHERE salary > ANY (SELECT salary FROM Employees WHERE dept_id = 20);  -- > 60000 (min of dept 20)

-- ALL: true if condition holds for every returned value
SELECT emp_name, salary FROM Employees
WHERE salary > ALL (SELECT salary FROM Employees WHERE dept_id = 20);  -- > 85000 (max of dept 20)
```

Dry Run: `salary > ALL (SELECT salary FROM Employees WHERE dept_id = 20)`

Dept 20 salaries = {85000, 60000} → ALL means salary must beat the maximum (85000).

emp_name	salary	> 85000?
Alice	90000	Yes ✓
Bob	75000	No
Carol	85000	No (not strictly greater)

Result: Alice only

9.4 Subquery in FROM clause (Derived Table)

```
SELECT dept_id, max_salary
FROM (
  SELECT dept_id, MAX(salary) AS max_salary
  FROM Employees
  GROUP BY dept_id
) AS dept_max
WHERE max_salary > 70000;
```

9.5 Correlated Subquery

A correlated subquery references a column from the outer query — it re-runs once per outer row (unlike a regular subquery, which runs once total).

```
-- Employees earning more than their own department's average
SELECT e1.emp_name, e1.salary, e1.dept_id
FROM Employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM Employees e2
    WHERE e2.dept_id = e1.dept_id  -- correlation: references outer e1
);
```

Dry Run

dept_id	dept avg	Qualifying employees
10	82500	Alice (90000 > 82500)
20	72500	Carol (85000 > 72500)
30	75000	none (Eve = avg exactly, not >)

Result: Alice, Carol

9.6 EXISTS / NOT EXISTS

EXISTS returns true as soon as the subquery finds at least one row — usually faster than IN for large, correlated checks, and correctly handles NULLs (unlike NOT IN).

```
-- Departments that have at least one employee
SELECT d.dept_name FROM Departments d
WHERE EXISTS (SELECT 1 FROM Employees e WHERE e.dept_id = d.dept_id);

-- Departments with NO employees (note: same result as the LEFT JOIN version in section 8.8)
SELECT d.dept_name FROM Departments d
WHERE NOT EXISTS (SELECT 1 FROM Employees e WHERE e.dept_id = d.dept_id);
```

NOT IN pitfall: if the subquery's result contains even a single NULL, **NOT IN** returns an empty result set for the whole query (because comparing to NULL is UNKNOWN, not TRUE). **NOT EXISTS** does not have this problem — prefer it when the subquery column can contain NULLs.

```
-- DANGEROUS if manager_id has NULLs (it does — Alice's manager_id is NULL)
SELECT emp_name FROM Employees
WHERE emp_id NOT IN (SELECT manager_id FROM Employees);  -- may return 0 rows!

-- SAFE version
SELECT emp_name FROM Employees e
WHERE NOT EXISTS (SELECT 1 FROM Employees m WHERE m.manager_id = e.emp_id);
```

9.7 Nested / Multiple Subqueries (subquery inside a subquery)

```
-- Employees who work in the department that has the company's highest average salary
SELECT emp_name, salary, dept_id
FROM Employees
WHERE dept_id = (
    SELECT dept_id FROM (
        SELECT dept_id, AVG(salary) AS avg_sal
        FROM Employees
        GROUP BY dept_id
    ) AS dept_avgs
    ORDER BY avg_sal DESC
    LIMIT 1
);
```

Dry Run (inside-out)

- Step 1 — innermost: group averages → {10:82500, 20:72500, 30:75000, NULL:55000}
- Step 2 — order by avg_sal DESC, take top 1 → dept_id = 10
- Step 3 — outer query: employees with dept_id = 10 → **Alice, Bob**

9.8 Subquery Types Summary

Type	Returns	Typical clause	Runs
Scalar	Single value	WHERE, SELECT	Once
Multi-row	List of values	WHERE ... IN/ANY/ALL	Once
Derived table	A full result set	FROM	Once
Correlated	Depends on outer row	WHERE, SELECT	Once per outer row

EXISTS	Boolean (row found or not)	WHERE	Once per outer row (short-circuits)
--------	----------------------------	-------	-------------------------------------

10. Set Operations

Combine results of two SELECT statements. Both must have the same number of columns with compatible data types.

Operator	Returns	Removes duplicates?
UNION	Rows from either query	Yes
UNION ALL	Rows from either query	No — keeps duplicates, faster
INTERSECT	Rows present in both queries	Yes
EXCEPT (MINUS in Oracle)	Rows in first query but not in second	Yes

```
-- All distinct locations that are either a department location or a client location
SELECT location FROM Departments
UNION
SELECT city AS location FROM Clients;

-- Keep duplicates (faster, no dedup step)
SELECT dept_id FROM Employees
UNION ALL
SELECT dept_id FROM Archive_Employees;

-- Departments that have BOTH engineering and sales employees assigned (example concept)
SELECT dept_id FROM Employees WHERE hire_date < '2021-01-01'
INTERSECT
SELECT dept_id FROM Employees WHERE salary > 70000;

-- dept_ids present in Employees but never in Projects
SELECT dept_id FROM Employees
EXCEPT
SELECT dept_id FROM Projects;
```

Interview favorite: "UNION vs UNION ALL" — UNION performs an implicit DISTINCT sort/hash pass to remove duplicates (slower on large data); UNION ALL simply concatenates result sets (faster). Use UNION ALL whenever you know there won't be duplicates, or duplicates are acceptable.

11. Window Functions

Window functions compute a value across a "window" (set) of rows related to the current row, **without collapsing rows** the way GROUP BY does. General syntax:

```
function_name(...) OVER (
  PARTITION BY col1      -- optional: split into groups
  ORDER BY col2          -- optional: order within each group
  ROWS/RANGE BETWEEN ... -- optional: frame boundaries
)
```

11.1 ROW_NUMBER, RANK, DENSE_RANK

```
SELECT emp_name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
       RANK()          OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rnk
FROM Employees;
```

Dry Run (dept 10: Alice 90000, Bob 75000 — dept 20: Carol 85000, David 60000)

emp_name	dept_id	salary	row_num	rank	dense_rank
Alice	10	90000	1	1	1
Bob	10	75000	2	2	2
Carol	20	85000	1	1	1
David	20	60000	2	2	2

Key difference (classic interview question): for tied values — ROW_NUMBER always gives unique increasing numbers (1,2,3,4...) even for ties; RANK gives the same rank to ties but **skips** the next rank (1,1,3,4); DENSE_RANK gives the same rank to ties with **no gap** (1,1,2,3).

Tie example: salaries 90000, 85000, 85000, 70000

salary	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
85000	2	2	2
85000	3	2	2
70000	4	4	3

11.2 LEAD and LAG

Access the next / previous row's value without a self-join.

```
SELECT emp_name, hire_date,
       LAG(emp_name) OVER (ORDER BY hire_date) AS hired_before_me,
       LEAD(emp_name) OVER (ORDER BY hire_date) AS hired_after_me
FROM Employees;
```

Dry Run (ordered by hire_date)

emp_name	hire_date	LAG (prev)	LEAD (next)
Carol	2019-07-01	NULL	Alice
Alice	2020-01-15	Carol	Bob
Bob	2021-03-10	Alice	Eve
Eve	2021-11-30	Bob	David
David	2022-05-20	Eve	Frank
Frank	2023-02-14	David	NULL

11.3 Aggregate Window Functions — Running Total, Moving Average

```
-- Running total of salary ordered by hire_date
SELECT emp_name, hire_date, salary,
       SUM(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
```

```
FROM Employees;

-- 3-row moving average
SELECT emp_name, hire_date, salary,
       AVG(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg_3
FROM Employees;
```

Dry Run: Running Total (ordered by hire_date)

emp_name	salary	running_total
Carol	85000	85000
Alice	90000	175000
Bob	75000	250000
Eve	75000	325000
David	60000	385000
Frank	55000	440000

11.4 NTILE — Bucketing rows

```
-- Split employees into 2 salary buckets (high half / low half)
SELECT emp_name, salary, NTILE(2) OVER (ORDER BY salary DESC) AS bucket
FROM Employees;
```

11.5 FIRST_VALUE / LAST_VALUE

```
SELECT emp_name, dept_id, salary,
       FIRST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS top_earner_in_dept
FROM Employees;
```

11.6 Nth Highest Value Using Window Functions

```
-- 2nd highest salary overall
SELECT emp_name, salary FROM (
  SELECT emp_name, salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employees
) ranked
WHERE rnk = 2;
-- Result: Carol, 85000
```

Why DENSE_RANK, not ROW_NUMBER, for "Nth highest"? If two employees tie for the highest salary, ROW_NUMBER would call one of them "1" and the other "2" — incorrectly treating the tied value as both 1st AND 2nd highest. DENSE_RANK correctly gives both the same rank so "2nd highest" means the next *distinct* value.

12. Common Table Expressions (CTE)

A CTE is a named temporary result set defined with `WITH`, readable only within the statement that follows it. It makes complex queries with nested subqueries far more readable.

12.1 Basic CTE

```
WITH DeptAvg AS (  
  SELECT dept_id, AVG(salary) AS avg_salary  
  FROM Employees  
  GROUP BY dept_id  
)  
SELECT e.emp_name, e.salary, d.avg_salary  
FROM Employees e  
JOIN DeptAvg d ON e.dept_id = d.dept_id  
WHERE e.salary > d.avg_salary;
```

12.2 Multiple CTEs

```
WITH DeptAvg AS (  
  SELECT dept_id, AVG(salary) AS avg_salary FROM Employees GROUP BY dept_id  
) ,  
HighAvgDepts AS (  
  SELECT dept_id FROM DeptAvg WHERE avg_salary > 70000  
)  
SELECT e.emp_name, e.dept_id  
FROM Employees e  
WHERE e.dept_id IN (SELECT dept_id FROM HighAvgDepts);
```

12.3 Recursive CTE

Used for hierarchical / graph-like data — e.g. building an org chart from manager_id.

```
WITH RECURSIVE OrgChart AS (  
  -- Anchor member: top-level employee (no manager)  
  SELECT emp_id, emp_name, manager_id, 1 AS level  
  FROM Employees  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- Recursive member: join next level to the previous result  
  SELECT e.emp_id, e.emp_name, e.manager_id, oc.level + 1  
  FROM Employees e  
  JOIN OrgChart oc ON e.manager_id = oc.emp_id  
)  
SELECT * FROM OrgChart ORDER BY level;
```

Dry Run

Iteration	New rows added	level
Anchor	Alice (manager_id NULL)	1
Recursive #1	Bob, Carol, Frank (manager_id = 1)	2
Recursive #2	David, Eve (manager_id = 3, Carol's id)	3
Recursive #3	no new rows → stop	—

Final result: Alice(1) → Bob,Carol,Frank(2) → David,Eve(3)

Interview favorite use case: recursive CTEs solve "find all subordinates of a manager", "generate a date series", or "explode a bill-of-materials tree" — anywhere the depth of nesting is unknown.

13. Views

A view is a saved, named SELECT statement — a virtual table that doesn't store data itself (unless materialized).

```
CREATE VIEW HighEarnersView AS
SELECT emp_name, salary, dept_id
FROM Employees
WHERE salary > 70000;

SELECT * FROM HighEarnersView;      -- query it like a table

CREATE OR REPLACE VIEW HighEarnersView AS
SELECT emp_name, salary, dept_id FROM Employees WHERE salary > 75000;

DROP VIEW HighEarnersView;

-- Materialized view (stores actual data, must be refreshed) - PostgreSQL/Oracle
CREATE MATERIALIZED VIEW DeptSummary AS
SELECT dept_id, COUNT(*) AS emp_count, AVG(salary) AS avg_salary
FROM Employees GROUP BY dept_id;
REFRESH MATERIALIZED VIEW DeptSummary;
```

View	Materialized View
No storage — runs underlying query each time	Stores actual result data on disk
Always up to date	Needs manual/scheduled REFRESH
Slower for complex/heavy queries	Fast reads, ideal for reporting/dashboards

Interview favorite: "Can you update data through a view?" Yes, if the view is simple (single table, no GROUP BY/aggregates/DISTINCT/joins) — it's then called an *updatable view*. Complex views are read-only.

14. Indexes & Query Performance

An index is a data structure (typically a B-Tree) that speeds up row lookups at the cost of extra storage and slower writes (every INSERT/UPDATE/DELETE must also update the index).

```
CREATE INDEX idx_emp_dept ON Employees(dept_id);
CREATE UNIQUE INDEX idx_emp_email ON Employees(email);
CREATE INDEX idx_emp_dept_salary ON Employees(dept_id, salary);  -- composite index
DROP INDEX idx_emp_dept;

-- Check how a query plans to use indexes
EXPLAIN SELECT * FROM Employees WHERE dept_id = 10;
```

Index type	Use case
Clustered	Physically sorts table data by this key; only one per table (often the PK)
Non-clustered	Separate structure pointing back to rows; many allowed per table
Unique	Enforces uniqueness in addition to speeding lookups
Composite	Covers multiple columns; column order matters (leftmost-prefix rule)

Interview favorite: "When would you NOT add an index?" On small tables, on columns rarely used in WHERE/JOIN/ORDER BY, on columns with very low cardinality (e.g. a boolean flag), or on tables with very heavy write traffic where index-maintenance overhead outweighs read benefits.

15. Stored Procedures & User Defined Functions

15.1 Stored Procedure

```
DELIMITER //
CREATE PROCEDURE GetEmployeesByDept(IN p_dept_id INT)
BEGIN
    SELECT emp_name, salary
    FROM Employees
    WHERE dept_id = p_dept_id;
END //
DELIMITER ;

CALL GetEmployeesByDept(10);
```

15.2 User Defined Function (UDF)

```
DELIMITER //
CREATE FUNCTION AnnualSalary(monthly DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN monthly * 12;
END //
DELIMITER ;

SELECT emp_name, AnnualSalary(salary) AS yearly_salary FROM Employees;
```

Stored Procedure	Function
May or may not return a value; can return multiple result sets	Must return exactly one value (or table)
Called with CALL	Used directly inside a SELECT/WHERE expression
Can perform DML (INSERT/UPDATE/DELETE)	Generally read-only / side-effect free
Cannot be used inside a SELECT list	Can be used inside a SELECT list

16. Triggers

A trigger is code that automatically runs in response to INSERT, UPDATE, or DELETE on a table.

```
CREATE TABLE Salary_Audit (  
    emp_id INT, old_salary DECIMAL(10,2), new_salary DECIMAL(10,2), changed_at TIMESTAMP  
);  
  
DELIMITER //  
CREATE TRIGGER trg_salary_update  
AFTER UPDATE ON Employees  
FOR EACH ROW  
BEGIN  
    IF NEW.salary <> OLD.salary THEN  
        INSERT INTO Salary_Audit(emp_id, old_salary, new_salary, changed_at)  
        VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());  
    END IF;  
END //  
DELIMITER ;
```

Timing	Event	Common use
BEFORE	INSERT / UPDATE	Validate or auto-fill values before they're written
AFTER	INSERT / UPDATE / DELETE	Audit logging, cascading updates, notifications

Triggers are powerful but easy to overuse — hidden logic in triggers makes debugging harder since the behavior isn't visible in the application code that issued the original statement.

17. Transactions & ACID Properties

A transaction is a group of SQL statements executed as a single unit of work — either all succeed (COMMIT) or all are undone (ROLLBACK).

```
BEGIN TRANSACTION;

UPDATE Employees SET salary = salary - 5000 WHERE emp_id = 1;
UPDATE Employees SET salary = salary + 5000 WHERE emp_id = 2;

-- If both succeeded:
COMMIT;
-- If something went wrong:
ROLLBACK;

-- Partial rollback using savepoints
BEGIN TRANSACTION;
UPDATE Employees SET salary = salary + 1000 WHERE dept_id = 10;
SAVEPOINT sp1;
UPDATE Employees SET salary = salary - 500 WHERE dept_id = 20;
ROLLBACK TO sp1;    -- undoes only the dept_id = 20 update
COMMIT;
```

ACID Properties

Property	Meaning
Atomicity	All statements in a transaction succeed together or fail together — no partial updates
Consistency	A transaction moves the database from one valid state to another, respecting all constraints
Isolation	Concurrent transactions don't interfere with each other's intermediate state
Durability	Once committed, changes survive even a system crash

Isolation Levels (lowest to highest isolation)

Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Interview favorite: "Dirty read vs Non-repeatable read vs Phantom read" — Dirty read: reading another transaction's *uncommitted* change. Non-repeatable read: re-reading the same row gives a *different value* because another transaction committed an update in between. Phantom read: re-running the same query returns *new/missing rows* because another transaction inserted/deleted rows matching the filter.

18. Normalization

Normalization organizes tables to reduce data redundancy and avoid update/insert/delete anomalies.

Form	Rule
1NF	Each column holds atomic (indivisible) values; no repeating groups or arrays in a cell
2NF	1NF + every non-key column depends on the whole primary key (no partial dependency on composite keys)
3NF	2NF + no non-key column depends on another non-key column (no transitive dependency)
BCNF	Stronger 3NF: every determinant must be a candidate key

Example: Un-normalized → 1NF

Bad: a single row with `subjects = "Math, Science"` (multiple values in one cell). Fix: one row per (student, subject) pair.

Example: 1NF → 2NF

Table `(order_id, product_id, product_name, quantity)` with composite key `(order_id, product_id)`: `product_name` depends only on `product_id`, not the full key → move it to a separate Products table.

Example: 2NF → 3NF

Table `(emp_id, dept_id, dept_name)`: `dept_name` depends on `dept_id`, which is a non-key column, not directly on `emp_id` (the primary key) → transitive dependency. Fix: move `dept_name` to a separate Departments table (exactly like our sample schema in Section 1).

Interview favorite: "Why normalize, and when would you denormalize?" Normalize to eliminate redundancy and anomalies. Denormalize (intentionally add redundancy) for read-heavy analytics/reporting systems where join costs matter more than storage or write-time consistency.

19. String, Date & Numeric Functions

19.1 String Functions

```
SELECT UPPER(emp_name), LOWER(emp_name) FROM Employees;
SELECT LENGTH(emp_name) FROM Employees;           -- CHAR_LENGTH() also common
SELECT CONCAT(emp_name, ' - ', dept_id) FROM Employees;
SELECT emp_name || '-' || dept_id FROM Employees;  -- PostgreSQL/Oracle concat operator
SELECT SUBSTRING(emp_name, 1, 3) FROM Employees;   -- first 3 chars
SELECT TRIM(' hello ');                             -- 'hello'
SELECT REPLACE(emp_name, 'a', '@') FROM Employees;
SELECT LEFT(emp_name, 2), RIGHT(emp_name, 2) FROM Employees;
SELECT LPAD(emp_id, 5, '0') FROM Employees;         -- '00001'
```

19.2 Date Functions

```
SELECT NOW();                                     -- current date + time
SELECT CURDATE();                                -- current date (MySQL)
SELECT DATEDIFF(NOW(), hire_date) AS days_employed FROM Employees;
SELECT DATE_ADD(hire_date, INTERVAL 1 YEAR) FROM Employees;
SELECT YEAR(hire_date), MONTH(hire_date), DAY(hire_date) FROM Employees;
SELECT EXTRACT(YEAR FROM hire_date) FROM Employees; -- ANSI standard
SELECT DATE_FORMAT(hire_date, '%d-%m-%Y') FROM Employees; -- MySQL formatting
```

19.3 Numeric & Conditional Functions

```
SELECT ROUND(salary, -3) FROM Employees;           -- round to nearest 1000
SELECT CEIL(salary / 1000), FLOOR(salary / 1000) FROM Employees;
SELECT ABS(-25);

-- COALESCE: first non-NULL value
SELECT emp_name, COALESCE(manager_id, 0) AS manager_or_zero FROM Employees;

-- NULLIF: returns NULL if both args are equal, else first arg
SELECT NULLIF(dept_id, 0) FROM Employees;

-- CASE expression
SELECT emp_name, salary,
       CASE
         WHEN salary >= 80000 THEN 'High'
         WHEN salary >= 65000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_band
FROM Employees;
```

Dry Run: CASE salary_band

emp_name	salary	salary_band
Alice	90000	High
Carol	85000	High
Bob	75000	Medium
Eve	75000	Medium
David	60000	Low
Frank	55000	Low

20. Most Repeated SQL Interview Questions

25 of the most frequently asked SQL interview questions, each with a working solution. All use the Employees / Departments tables from Section 1 unless noted.

Q1

Find the Nth highest salary

```
-- Method 1: DENSE_RANK (handles ties correctly, works everywhere)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM Employees
) t WHERE rnk = 3;

-- Method 2: LIMIT / OFFSET (MySQL/PostgreSQL) - no ties handling
SELECT DISTINCT salary FROM Employees ORDER BY salary DESC LIMIT 1 OFFSET 2; -- N=3 -> OFFSET N-1

-- Method 3: Correlated subquery (portable to almost any SQL engine)
SELECT DISTINCT salary FROM Employees e1
WHERE 2 = (SELECT COUNT(DISTINCT salary) FROM Employees e2 WHERE e2.salary > e1.salary); -- N=3 -> use N-1
```

Distinct salaries sorted desc: 90000, 85000, 75000, 60000, 55000 → 3rd highest = **75000**

Q2

Find the second highest salary (classic special case of Q1)

```
SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
-- Result: 85000
```

Q3

Find duplicate rows in a table

```
-- Suppose Employees had duplicate (emp_name, salary) entries
SELECT emp_name, salary, COUNT(*) AS occurrences
FROM Employees
GROUP BY emp_name, salary
HAVING COUNT(*) > 1;
```

Q4

Delete duplicate rows, keeping only one copy

```
-- Keep the row with the smallest emp_id per duplicate group
DELETE e1 FROM Employees e1
JOIN Employees e2
    ON e1.emp_name = e2.emp_name
   AND e1.salary = e2.salary
   AND e1.emp_id > e2.emp_id;

-- Portable version using ROW_NUMBER (PostgreSQL/SQL Server/Oracle)
DELETE FROM Employees
WHERE emp_id IN (
    SELECT emp_id FROM (
        SELECT emp_id,
            ROW_NUMBER() OVER (PARTITION BY emp_name, salary ORDER BY emp_id) AS rn
        FROM Employees
    ) t WHERE rn > 1
);
```

Q5

Department-wise highest paid employee

```
SELECT dept_id, emp_name, salary FROM (
```

```

SELECT dept_id, emp_name, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
FROM Employees
) t WHERE rnk = 1;

```

Dry Run Result

dept_id	emp_name	salary
10	Alice	90000
20	Carol	85000
30	Eve	75000
NULL	Frank	55000

Q6

Employees earning more than their manager

```

SELECT e.emp_name AS employee, e.salary, m.emp_name AS manager, m.salary AS manager_salary
FROM Employees e
JOIN Employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;

```

Managers: Bob→Alice(90000), Carol→Alice(90000), David→Carol(85000), Eve→Carol(85000), Frank→Alice(90000). None of Bob/David/Eve/Frank's salaries exceed their manager's → **empty result** with this data (demonstrates the query still needs to be written correctly even when the answer set is empty).

Q7

Count of employees in each department, including departments with zero employees

```

SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY emp_count DESC;

```

Q8

Employees who joined in the last N days (e.g. last 365 days from a reference date)

```

SELECT emp_name, hire_date
FROM Employees
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 365 DAY);    -- MySQL
-- PostgreSQL: WHERE hire_date >= CURRENT_DATE - INTERVAL '365 days'
-- SQL Server: WHERE hire_date >= DATEADD(DAY, -365, GETDATE())

```

Q9

Find duplicate email addresses (classic LeetCode-style)

```

-- Table: Person(id, email)
SELECT email, COUNT(*) AS cnt
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;

```

Q10

Three (or more) consecutive numbers/rows with the same value

Classic problem: find stadium days with 3+ consecutive rows where `people >= 100`.

```

-- Table: Stadium(id, visit_date, people)  -- id is consecutive when people>=100
SELECT DISTINCT s1.*
FROM Stadium s1, Stadium s2, Stadium s3
WHERE s1.people >= 100 AND s2.people >= 100 AND s3.people >= 100

```

```

AND (
    (s1.id = s2.id - 1 AND s2.id = s3.id - 1)    -- s1,s2,s3 consecutive
    OR (s2.id = s1.id - 1 AND s1.id = s3.id - 1)  -- s2,s1,s3 consecutive
    OR (s1.id = s3.id - 1 AND s3.id = s2.id - 1)  -- s1,s3,s2 consecutive
)
ORDER BY s1.id;

-- Cleaner window-function version
SELECT id, visit_date, people FROM (
    SELECT *, id - ROW_NUMBER() OVER (ORDER BY id) AS grp
    FROM Stadium
    WHERE people >= 100
) t
GROUP BY grp HAVING COUNT(*) >= 3;

```

Gaps-and-islands trick: `id - ROW_NUMBER()` stays constant for any run of consecutive ids — this is the standard technique for finding "streaks" or "consecutive groups" in SQL.

Q11

Find the Nth row of a table (without a primary key ordering guarantee)

```

SELECT * FROM Employees ORDER BY emp_id LIMIT 1 OFFSET 4;  -- 5th row (0-indexed offset)

-- Using ROW_NUMBER (portable)
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (ORDER BY emp_id) AS rn FROM Employees
) t WHERE rn = 5;

```

Q12

Top N employees per department (Top-N-per-group)

```

-- Top 2 highest paid employees in each department
SELECT dept_id, emp_name, salary FROM (
    SELECT dept_id, emp_name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM Employees
) t
WHERE rnk <= 2;

```

Q13

Cumulative (running) sum of salary per department

```

SELECT emp_name, dept_id, salary,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM Employees;

```

Q14

Find employees who joined on the same date as another employee

```

SELECT e1.emp_name, e2.emp_name, e1.hire_date
FROM Employees e1
JOIN Employees e2 ON e1.hire_date = e2.hire_date AND e1.emp_id < e2.emp_id;

```

Q15

Pivot rows into columns (row-to-column transformation)

```

-- Show total salary per department as columns using conditional aggregation
SELECT
    SUM(CASE WHEN dept_id = 10 THEN salary ELSE 0 END) AS Engineering,
    SUM(CASE WHEN dept_id = 20 THEN salary ELSE 0 END) AS Sales,
    SUM(CASE WHEN dept_id = 30 THEN salary ELSE 0 END) AS Marketing
FROM Employees;

-- SQL Server native PIVOT

```

```
SELECT * FROM (
    SELECT dept_id, salary FROM Employees
) src
PIVOT (SUM(salary) FOR dept_id IN ([10],[20],[30])) AS pvt;
```

Q16

Find missing IDs in a sequence (gaps)

```
-- Employees table has emp_id 1-6 but suppose 3 is missing; generate a sequence and use EXCEPT/NOT IN
WITH RECURSIVE seq AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM seq WHERE n < (SELECT MAX(emp_id) FROM Employees)
)
SELECT n AS missing_id FROM seq
WHERE n NOT IN (SELECT emp_id FROM Employees);
```

Q17

Find employees with the same salary (salary "twins")

```
SELECT e1.emp_name, e2.emp_name, e1.salary
FROM Employees e1
JOIN Employees e2 ON e1.salary = e2.salary AND e1.emp_id < e2.emp_id;
-- Result: Bob & Eve, both 75000
```

Q18

Find the maximum salary without using the MAX() function

```
SELECT salary FROM Employees e1
WHERE NOT EXISTS (
    SELECT 1 FROM Employees e2 WHERE e2.salary > e1.salary
);
-- or: ORDER BY salary DESC LIMIT 1
```

Q19

Calculate each employee's salary as a percentage of total department salary

```
SELECT emp_name, dept_id, salary,
    ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept_id), 2) AS pct_of_dept
FROM Employees
WHERE dept_id IS NOT NULL;
```

Dry Run (dept 10 total = 165000)

emp_name	salary	pct_of_dept
Alice	90000	54.55%
Bob	75000	45.45%

Q20

Find the median salary

```
-- PostgreSQL / Oracle
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM Employees;

-- Portable version (works everywhere) - average of middle value(s)
SELECT AVG(salary) AS median_salary FROM (
    SELECT salary,
        ROW_NUMBER() OVER (ORDER BY salary) AS rn,
        COUNT(*) OVER () AS cnt
    FROM Employees
) t
WHERE rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2));
```

6 salaries sorted: 55000,60000,75000,75000,85000,90000 → middle two = 75000,75000 → **median = 75000**

Q21

Swap two column values without using a temporary variable

```
-- Swap 'M'/'F' in a single UPDATE using CASE
UPDATE Employees
SET gender = CASE gender WHEN 'M' THEN 'F' ELSE 'M' END;
```

Q22

Find employees whose name appears more than once (case-insensitive)

```
SELECT LOWER(emp_name) AS name, COUNT(*) AS cnt
FROM Employees
GROUP BY LOWER(emp_name)
HAVING COUNT(*) > 1;
```

Q23

Find the department with the highest total salary spend

```
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM Departments d
JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY total_salary DESC
LIMIT 1;
-- Result: Engineering, 165000
```

Q24

Multiple subqueries in a single statement (combining several patterns learned above)

```
-- Employees who (a) earn more than their department average AND
-- (b) belong to the department with the most employees
SELECT e.emp_name, e.salary, e.dept_id
FROM Employees e
WHERE e.salary > (
    SELECT AVG(e2.salary) FROM Employees e2 WHERE e2.dept_id = e.dept_id -- correlated subquery
)
AND e.dept_id = (
    SELECT dept_id FROM (
        SELECT dept_id, COUNT(*) AS cnt FROM Employees
        WHERE dept_id IS NOT NULL
        GROUP BY dept_id
        ORDER BY cnt DESC
        LIMIT 1
    ) busiest_dept -- subquery in FROM
);
```

Busiest departments (10 and 20 tie at 2 employees each — LIMIT 1 picks one, e.g. dept 10). Dept 10 average = 82500 → only Alice (90000) exceeds it → **Result: Alice**

Q25

Find employees who are managers (appear as someone else's manager_id)

```
SELECT DISTINCT m.emp_name
FROM Employees e
JOIN Employees m ON e.manager_id = m.emp_id;
-- Result: Alice, Carol

-- Using EXISTS instead of JOIN+DISTINCT
SELECT emp_name FROM Employees m
WHERE EXISTS (SELECT 1 FROM Employees e WHERE e.manager_id = m.emp_id);
```

21. Clause Execution Order & Final Cheat Sheet

21.1 Logical Order of Execution (not the order you write it!)

Step	Clause	Purpose
1	FROM	Identify source table(s)
2	JOIN ... ON	Combine tables
3	WHERE	Filter individual rows
4	GROUP BY	Group rows
5	HAVING	Filter groups
6	SELECT	Compute output columns / expressions
7	DISTINCT	Remove duplicate output rows
8	ORDER BY	Sort the result
9	LIMIT / OFFSET	Restrict row count

Why this matters: it explains why you can't reference a SELECT alias in WHERE (WHERE runs before SELECT), but you *can* reference it in ORDER BY (which runs after SELECT). It also explains why aggregate functions are allowed in HAVING but not WHERE.

21.2 Written (Syntax) Order vs Execution Order

SELECT	...	-- written 1st,	executed 6th
FROM	...	-- written 2nd,	executed 1st
JOIN	...	-- written 3rd,	executed 2nd
WHERE	...	-- written 4th,	executed 3rd
GROUP BY	...	-- written 5th,	executed 4th
HAVING	...	-- written 6th,	executed 5th
ORDER BY	...	-- written 7th,	executed 8th
LIMIT	...	-- written 8th,	executed 9th

21.3 Quick Reference: Which Tool for Which Job

Need	Use
Compare row to an aggregate of itself/its group	Correlated subquery or window function
Keep all rows but rank/number them	Window function (ROW_NUMBER/RANK/DENSE_RANK)
Collapse rows into groups	GROUP BY + aggregate functions
Combine two full result sets vertically	UNION / UNION ALL
Combine columns from related tables	JOIN
Reusable, readable intermediate result	CTE (WITH clause)
Hierarchical / unknown-depth data	Recursive CTE
Row exists check, NULL-safe	EXISTS / NOT EXISTS (avoid NOT IN with nullable columns)
Streaks / consecutive groups	id – ROW_NUMBER() "gaps and islands" trick

21.4 Most Common Interview Traps — Quick Recap

- `WHERE` cannot use aggregate functions — use `HAVING` instead.
- `NOT IN` with a NULL-containing subquery silently returns zero rows — use `NOT EXISTS`.
- `RIGHT JOIN` can always be rewritten as a `LEFT JOIN` by swapping table order — most style guides prefer LEFT JOIN for readability.
- Use `DENSE_RANK()`, not `ROW_NUMBER()`, when ties should share the same "Nth highest" rank.
- `UNION` removes duplicates (slower); `UNION ALL` keeps them (faster) — don't use UNION out of habit.
- `DELETE` is DML and rollback-able; `TRUNCATE` and `DROP` are DDL and typically cannot be rolled back.
- A view built on a JOIN/GROUP BY/DISTINCT is usually not updatable — only simple single-table views are.

Final tip: Whenever an interview question says "for each group", think `GROUP BY` if rows should collapse, or a **window function** if individual rows must be preserved. Whenever it says "compare against some aggregate of the same row's group", think **correlated subquery** or `OVER (PARTITION BY ...)`.